

UNIVERSIDAD DE SANTIAGO DE CHILE
FACULTAD DE ADMINISTRACIÓN Y ECONOMÍA

Portafolio de Inversiones Gestionado por Inteligencia Artificial

*Trabajo de Graduación para optar al Grado Académico de
Magíster en Ciencia de Datos*

Autor: Daniel Alejandro Carrasco Urrutia

Profesor Guía: Javier Espinosa Brito

Santiago – Chile, 2026

Resumen

La gestión de portafolios de inversión en el mercado accionario representa un desafío de alta complejidad para el inversor individual, quien enfrenta un volumen creciente de información de mercado, volatilidad no lineal y limitaciones en su capacidad de monitoreo continuo. El presente proyecto de título desarrolla una aplicación web de gestión de portafolio de inversiones administrada por inteligencia artificial, que automatiza el análisis, la toma de decisiones y el rebalanceo de una cartera de acciones del mercado estadounidense, operando en modalidad de *paper trading* (simulación con datos reales).

El sistema integra dos ejes tecnológicos complementarios. En el ámbito del *machine learning*, se implementan y evalúan comparativamente cuatro modelos de predicción de series de tiempo financieras: LSTM (Long Short-Term Memory), XGBoost, Prophet y Random Forest, entrenados con datos históricos de Yahoo Finance. La evaluación se realiza mediante *backtesting* con validación *walk-forward*, sin sesgo de anticipación (*look-ahead bias*): el clasificador de riesgo Random Forest alcanza un AUC ROC global de 78,2%, mientras que la señal de compra/venta de XGBoost se sitúa en torno al azar (53,6%), resultado consistente con la hipótesis de eficiencia débil del mercado. En el ámbito de la inteligencia artificial, un agente autónomo consolida las cuatro predicciones en una decisión determinista y auditable de rebalanceo, cuya justificación se redacta en lenguaje natural mediante modelos de lenguaje de gran escala (Claude, de Anthropic), con orquestación multi-motor basada en OpenClaw y canal de interacción vía Telegram.

La arquitectura es distribuida y se encuentra desplegada y operativa: frontend en React con Tailwind CSS sobre Vercel, backend RESTful en FastAPI sobre Railway, base de datos PostgreSQL en Supabase y automatización diaria mediante GitHub Actions. La conectividad con el mercado para ejecución simulada de órdenes se realiza mediante la API de Alpaca en modalidad *paper trading*.

El resultado es un sistema funcional en producción que demuestra la integración efectiva de herramientas de IA y modelos de ML en la gestión automatizada de portafolios, constituyendo una contribución práctica en la intersección de la ciencia de datos, las finanzas computacionales y el desarrollo de software.

Palabras clave: portafolio de inversiones, inteligencia artificial agéntica, machine learning, agente autónomo, predicción financiera, LSTM, XGBoost, Prophet, Random Forest, backtesting walk-forward, trading algorítmico.

Abstract

Investment portfolio management in equity markets poses a high-complexity challenge for individual investors, who must navigate an ever-growing volume of market information, non-linear volatility, and the limitations of continuous monitoring. This thesis project develops an AI-managed investment portfolio web application that automates the analysis, decision-making, and rebalancing of a U.S. stock portfolio, operating in paper trading mode (simulation over real market data).

The system integrates two complementary technological axes. In the machine learning domain, four financial time series prediction models are implemented and comparatively evaluated: LSTM, XGBoost, Prophet, and Random Forest, trained on historical data from Yahoo Finance. Evaluation follows a walk-forward backtesting methodology free of look-ahead bias: the Random Forest risk classifier achieves a global ROC AUC of 78.2%, while the XGBoost buy/sell signal performs near chance level (53.6%), a result consistent with the weak-form market efficiency hypothesis. In the artificial intelligence domain, an autonomous agent consolidates the four predictions into a deterministic, auditable rebalancing decision, whose rationale is written in natural language by large language models (Anthropic's Claude), with multi-engine orchestration based on OpenClaw and a Telegram interaction channel.

The architecture is distributed and currently deployed and operational: a React + Tailwind CSS frontend on Vercel, a RESTful FastAPI backend on Railway, a PostgreSQL database on Supabase, and daily automation through GitHub Actions. Market connectivity for simulated order execution is provided by the Alpaca API in paper trading mode.

The result is a functional production system demonstrating the effective integration of AI tools and ML models in automated portfolio management, a practical contribution at the intersection of data science, computational finance, and software engineering.

Keywords: investment portfolio, agentic artificial intelligence, machine learning, autonomous agent, financial prediction, LSTM, XGBoost, Prophet, Random Forest, walk-forward

backtesting, algorithmic trading.

1. Introducción

La gestión de portafolios de inversión en el mercado accionario representa uno de los desafíos más complejos para el inversor contemporáneo. El crecimiento exponencial del volumen de datos financieros disponibles, la volatilidad no lineal de los mercados y la necesidad de monitoreo continuo configuran un escenario que excede ampliamente las capacidades cognitivas y operativas de un individuo actuando de forma manual. Históricamente, las herramientas de análisis sofisticado y la gestión algorítmica de carteras han sido patrimonio casi exclusivo de instituciones financieras y fondos de inversión con acceso a infraestructura tecnológica especializada, dejando al inversor individual con opciones limitadas y reactivas frente a las dinámicas del mercado.

En este contexto, el avance reciente de la inteligencia artificial —y en particular el surgimiento de los modelos de lenguaje de gran escala (LLM) y los sistemas de agentes autónomos— abre una ventana de oportunidad significativa. Los agentes de IA agéntica son capaces de percibir el entorno, razonar sobre múltiples fuentes de información, planificar acciones y ejecutarlas de forma iterativa sin intervención humana constante. Esta característica los convierte en candidatos naturales para automatizar la supervisión y la toma de decisiones en contextos financieros dinámicos, donde la velocidad de respuesta y la consistencia analítica son determinantes.

Sin embargo, la sola aplicación de IA conversacional no es suficiente. Las decisiones de inversión requieren fundamento cuantitativo: la capacidad de anticipar el comportamiento futuro de los precios de los activos a partir de datos históricos. Es en este punto donde los modelos de *machine learning* aplicados a series de tiempo financieras cumplen un rol complementario e indispensable. Algoritmos como LSTM, XGBoost, Prophet y Random Forest han demostrado capacidad para capturar patrones complejos en datos financieros, aportando señales predictivas que enriquecen el proceso de decisión del agente. Con igual importancia, la validez de esas señales exige una metodología de evaluación rigurosa: en este proyecto, todas las métricas reportadas provienen de *backtesting* con validación *walk-forward*, un esquema que reentrena los modelos por ventanas temporales y garantiza que cada predicción se construye exclusivamente con información disponible hasta ese momento, eliminando el sesgo de

anticipación que invalida buena parte de los resultados publicados en la literatura aplicada (López de Prado, 2018).

El presente proyecto de título desarrolla una aplicación web de gestión de portafolio de inversiones administrada por inteligencia artificial que integra ambas dimensiones tecnológicas en un sistema funcional, hoy desplegado y operativo en la nube. Cuatro modelos de *machine learning* entrenados con datos históricos de Yahoo Finance producen diariamente señales predictivas sobre los activos en cartera; un agente autónomo consolida esas señales en decisiones de rebalanceo deterministas y auditables, y las comunica en lenguaje natural mediante LLM (Claude de Anthropic, con orquestación multi-motor basada en OpenClaw y canal de mensajería Telegram); la conectividad con el mercado se realiza mediante la API de Alpaca en modalidad de *paper trading*, de modo que el sistema ejecuta sus órdenes en simulación sobre datos reales, sin riesgo de capital.

La hipótesis que articula el trabajo sostiene que la implementación de un agente de inteligencia artificial agéntica, que integra modelos de *machine learning* para la predicción de series de tiempo financieras y datos de mercado en tiempo real, permite administrar de manera automatizada un portafolio de inversiones en el mercado accionario, generando decisiones de rebalanceo coherentes con el comportamiento del mercado y las preferencias del usuario. En consecuencia, el objetivo general es demostrar que mediante el uso de IA agéntica, integrada con modelos predictivos de *machine learning*, es posible administrar de forma automatizada un portafolio de inversiones, proveyendo al usuario análisis continuo, señales de rebalanceo y un canal de interacción en lenguaje natural. De este objetivo se desprenden como objetivos específicos: (i) implementar y comparar los cuatro modelos predictivos sobre un universo de acciones estadounidenses; (ii) diseñar una metodología de evaluación sin sesgo de anticipación que permita defender las métricas obtenidas; (iii) construir el agente de consolidación de señales y su capa de explicación en lenguaje natural; y (iv) desplegar el sistema completo como aplicación web operativa con datos frescos diarios.

El documento se estructura de la siguiente manera: el capítulo 2 desarrolla el marco teórico que sustenta los componentes financieros y tecnológicos del sistema; el capítulo 3 describe la arquitectura implementada y la metodología de modelamiento y evaluación; el capítulo 4 presenta los resultados preliminares del *backtesting* y el estado del sistema en producción; el capítulo 5 detalla el plan de trabajo restante; y el capítulo 6 expone las conclusiones preliminares y las líneas de trabajo futuro.

2. Marco Teórico

2.1 Mercados financieros y gestión de portafolios

Los mercados financieros constituyen el entorno en el cual los agentes económicos intercambian activos con el objetivo de transferir recursos a través del tiempo y gestionar el riesgo. Dentro de estos mercados, el mercado bursátil ocupa un rol central al permitir la compra y venta de instrumentos de renta variable, cuyo valor fluctúa en función de las expectativas de los participantes, los fundamentos económicos de las empresas y las condiciones macroeconómicas globales. Los precios de estos activos se comportan como series de tiempo, es decir, secuencias de observaciones indexadas en el tiempo que exhiben dependencia temporal y, en general, características no estacionarias como tendencia y volatilidad variable (Box y Jenkins, 1976).

La gestión de portafolios de inversión surge como disciplina orientada a la selección y combinación óptima de activos financieros con el fin de maximizar el retorno esperado para un nivel de riesgo determinado o, equivalentemente, minimizar el riesgo para un nivel de retorno objetivo. El fundamento teórico de esta disciplina fue establecido por Harry Markowitz (1952), quien formalizó el concepto de diversificación a través de la teoría moderna de portafolios. Markowitz demostró que el riesgo de un portafolio no equivale a la suma de los riesgos individuales de sus componentes, sino que depende de la covarianza entre los activos que lo integran. De este modo, es posible construir portafolios que, para un mismo nivel de retorno esperado, presenten una varianza menor que la de cualquiera de sus activos individuales.

A partir de este principio, Markowitz definió la frontera eficiente como el conjunto de portafolios que maximizan el retorno esperado para cada nivel de riesgo posible. Formalmente, el problema de optimización se plantea como

$$\min_w w^\top \Sigma w \quad \text{s.a.} \quad w^\top \mu = \mu_p, \quad \sum_i w_i = 1$$

donde w es el vector de pesos del portafolio, Σ la matriz de covarianzas de los retornos, μ el vector de retornos esperados y μ_p el retorno objetivo del portafolio.

La aplicación práctica del modelo de Markowitz enfrenta, sin embargo, limitaciones importantes. La estimación de los parámetros —en particular la matriz de covarianzas— es

sensible al período de estimación y puede introducir errores significativos cuando se trabaja con un número elevado de activos. Además, el modelo asume retornos normalmente distribuidos y no considera costos de transacción ni restricciones operacionales propias de los mercados reales.

Estas limitaciones han motivado el desarrollo de enfoques alternativos y complementarios para la gestión de portafolios, entre los cuales destaca la incorporación de modelos de predicción para anticipar el comportamiento futuro de los precios. La capacidad de estimar con mayor precisión los retornos esperados y el riesgo de los activos permite mejorar la calidad de las decisiones de asignación de capital. En este contexto, el presente proyecto propone el uso de modelos de *machine learning* para la predicción de precios, señales y niveles de riesgo, y de agentes de inteligencia artificial para la gestión dinámica del portafolio, integrando ambos enfoques en una plataforma web de inversión automatizada.

2.2 Machine Learning aplicado a predicción financiera

El *machine learning* (ML) es una rama de la inteligencia artificial que desarrolla algoritmos capaces de aprender patrones a partir de datos históricos y generalizar ese aprendizaje para realizar predicciones sobre datos no observados previamente. A diferencia de los modelos econométricos clásicos, que requieren supuestos explícitos sobre la distribución de los datos y la forma funcional de las relaciones entre variables, los modelos de ML capturan relaciones no lineales y estructuras complejas de manera inductiva, sin necesidad de especificarlas a priori (Salas, 2004).

En el contexto financiero, el ML ha demostrado ser una herramienta especialmente útil para la predicción de series de tiempo de precios y retornos de activos. Los precios de instrumentos financieros son secuencias de observaciones temporalmente dependientes, lo que hace que los métodos de aprendizaje supervisado orientados a series de tiempo sean particularmente adecuados para su modelamiento. El proceso general de entrenamiento consiste en dividir los datos históricos disponibles en un conjunto de entrenamiento, utilizado para ajustar los parámetros del modelo, y un conjunto de prueba, utilizado para evaluar su capacidad predictiva sobre datos no vistos; la proporción habitual es de 80% para entrenamiento y 20% para prueba (Ying, 2019). En series de tiempo, no obstante, la partición aleatoria clásica es inadecuada, pues mezcla pasado y futuro; la alternativa correcta es la validación por ventanas temporales

ordenadas (*walk-forward*), que este proyecto adopta y que se formaliza en la sección 3.4 (Bergmeir y Benítez, 2012; López de Prado, 2018).

La calidad predictiva de los modelos de regresión se evalúa mediante métricas de error, siendo las más utilizadas la raíz del error cuadrático medio (RMSE) y el error absoluto medio (MAE):

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{t=1}^n (\hat{y}_t - y_t)^2}, \quad \text{MAE} = \frac{1}{n} \sum_{t=1}^n |\hat{y}_t - y_t|$$

donde $\hat{y}_t$ corresponde al valor predicho e y_t al valor real en el período t . Para las tareas de clasificación (señal de compra/venta, nivel de riesgo) se emplean métricas específicas — *accuracy*, F1 y AUC ROC— cuya definición y justificación se desarrollan junto a la metodología de evaluación. En el presente proyecto se emplean cuatro modelos de ML con características y fortalezas complementarias: LSTM, XGBoost, Prophet y Random Forest.

2.2.1 Redes neuronales LSTM

Las redes neuronales recurrentes del tipo Long Short-Term Memory (LSTM) fueron propuestas por Hochreiter y Schmidhuber (1997) como solución al problema del gradiente que se desvanece, que afecta a las redes recurrentes tradicionales cuando se intenta capturar dependencias de largo plazo en secuencias temporales. La arquitectura LSTM incorpora un mecanismo de memoria explícito basado en tres compuertas: la compuerta de olvido, que determina qué información del estado anterior se descarta; la compuerta de entrada, que controla qué nueva información se almacena; y la compuerta de salida, que regula qué parte del estado interno se transmite como salida de la celda. Este diseño permite que la red retenga información relevante durante períodos prolongados, lo que resulta especialmente valioso en series financieras donde los patrones de mediano y largo plazo influyen sobre el comportamiento futuro de los precios.

2.2.2 XGBoost

XGBoost (Extreme Gradient Boosting) es un algoritmo de *ensemble* basado en la construcción secuencial de árboles de decisión, donde cada árbol nuevo se entrena para corregir los errores residuales del conjunto de árboles anteriores (Chen y Guestrin, 2016). El proceso de *boosting* minimiza iterativamente una función de pérdida mediante descenso de gradiente en el espacio

de funciones, incorporando términos de regularización que controlan la complejidad del modelo y reducen el riesgo de sobreajuste. XGBoost destaca por su eficiencia computacional, su robustez frente a datos faltantes y su capacidad para manejar variables de entrada heterogéneas, incluyendo indicadores técnicos, variables macroeconómicas y características derivadas de los precios históricos. En aplicaciones financieras ha mostrado resultados competitivos en tareas de clasificación de dirección de precios y predicción de retornos.

2.2.3 Prophet

Prophet es un modelo de predicción de series de tiempo desarrollado por Taylor y Letham (2018) en Meta, diseñado para series con patrones estacionales múltiples y tendencias no lineales. El modelo descompone la serie en tres componentes aditivos: tendencia, estacionalidad y efectos de eventos especiales. La tendencia se modela mediante funciones lineales por tramos o funciones logísticas con puntos de cambio automáticos, mientras que la estacionalidad se representa mediante series de Fourier. Prophet es robusto ante datos faltantes y valores atípicos, y no requiere que la serie sea estacionaria como condición previa. En el contexto del presente proyecto, Prophet se emplea para capturar los patrones estacionales y tendencias de mediano plazo presentes en los precios de los activos.

2.2.4 Random Forest

Random Forest es un algoritmo de *ensemble* que construye un conjunto de árboles de decisión entrenados sobre submuestras aleatorias de los datos y subconjuntos aleatorios de las variables de entrada, promediando sus predicciones para producir una estimación final (Breiman, 2001). Este procedimiento de aleatorización reduce la varianza del modelo sin incrementar significativamente el sesgo, mejorando la capacidad de generalización respecto a un único árbol de decisión. Random Forest provee además una medida natural de importancia de variables, lo que facilita interpretar qué características del mercado contribuyen más a la predicción. Su resistencia al sobreajuste y su capacidad para manejar un gran número de variables de entrada lo hacen particularmente adecuado para la predicción financiera, donde el espacio de características puede ser extenso.

2.3 Inteligencia artificial aplicada a la gestión financiera

La inteligencia artificial (IA) comprende un conjunto de técnicas y paradigmas computacionales orientados a replicar capacidades cognitivas humanas como el razonamiento, la comprensión del lenguaje, la toma de decisiones y el aprendizaje adaptativo. En la última década, el avance de los modelos de lenguaje de gran escala (*Large Language Models*, LLM) ha representado un salto cualitativo en esta disciplina, habilitando sistemas capaces de comprender instrucciones en lenguaje natural, generar respuestas contextualizadas y ejecutar razonamientos complejos sobre dominios especializados (Brown et al., 2020).

2.3.1 Modelos de lenguaje de gran escala (LLM)

Los LLM son redes neuronales de arquitectura Transformer entrenadas sobre grandes corpus de texto mediante aprendizaje auto-supervisado. Su capacidad para procesar y generar lenguaje natural los convierte en interfaces naturales entre el usuario y sistemas computacionales complejos, eliminando la necesidad de interacciones estructuradas mediante formularios o comandos explícitos. Modelos como GPT-4 de OpenAI y Claude de Anthropic han demostrado capacidades sobresalientes en tareas de razonamiento, síntesis de información y generación de respuestas en dominios financieros, incluyendo el análisis de reportes corporativos, la interpretación de indicadores macroeconómicos y la evaluación de condiciones de mercado (López-Lira y Tang, 2023).

En el presente proyecto se utilizan múltiples motores de LLM —Claude de Anthropic, modelos de OpenAI y modelos locales mediante Ollama— orquestados a través de un agente central. Esta arquitectura multi-motor permite seleccionar el modelo más adecuado según la naturaleza de la tarea, optimizando tanto la calidad de las respuestas como los costos operacionales asociados al uso de APIs externas. Un principio de diseño central, desarrollado en la sección 3.5, es que el LLM no toma la decisión de inversión: la decisión se calcula de forma determinista a partir de las predicciones de los modelos ML, y el LLM se emplea exclusivamente para redactar su justificación en lenguaje natural, preservando la auditabilidad del sistema.

2.3.2 Agentes de inteligencia artificial

Un agente de inteligencia artificial es un sistema autónomo capaz de percibir su entorno, razonar sobre él y ejecutar acciones orientadas al cumplimiento de objetivos definidos, de manera iterativa y sin requerir intervención humana en cada paso del proceso (Russell y Norvig,

2020). A diferencia de un modelo de lenguaje estático que responde a consultas individuales, un agente puede planificar secuencias de acciones, utilizar herramientas externas como APIs y bases de datos, evaluar los resultados de sus acciones y ajustar su comportamiento en consecuencia.

En el contexto financiero, los agentes de IA permiten automatizar el ciclo completo de gestión de un portafolio: desde la recolección y análisis de datos de mercado, pasando por la consulta a modelos predictivos de ML, hasta la ejecución de órdenes de compra y venta a través de APIs de brokers. Este paradigma representa una evolución respecto a los sistemas de trading algorítmico tradicionales, que operaban sobre reglas estáticas predefinidas sin capacidad de adaptación al contexto. En el presente proyecto, el agente actúa como núcleo orquestador del sistema: recibe las predicciones de los modelos ML, evalúa las condiciones del mercado, incorpora las preferencias y restricciones del usuario, y produce decisiones de rebalanceo que se ejecutan en simulación mediante la API de Alpaca. La comunicación con el usuario se realiza a través de interfaces de mensajería como Telegram, lo que permite una interacción en lenguaje natural sin necesidad de acceder a la interfaz web.

2.3.3 Trading algorítmico y APIs de brokers

El trading algorítmico consiste en la ejecución automatizada de órdenes de compra y venta de activos financieros mediante algoritmos que operan sobre señales de mercado predefinidas o generadas dinámicamente. Este enfoque elimina la latencia y los sesgos emocionales asociados a la toma de decisiones humana, permitiendo además la operación continua durante las horas de mercado (Narang, 2013).

Las APIs de brokers como Alpaca proporcionan el puente entre el sistema de gestión y los mercados financieros, exponiendo *endpoints* REST que permiten consultar precios en tiempo real, obtener el estado del portafolio, enviar órdenes de mercado o limitadas, y acceder a datos históricos. Alpaca ofrece además un entorno de *paper trading* para el desarrollo y validación de estrategias sin riesgo de capital real, modalidad que este proyecto adopta como alcance definitivo: el sistema ejecuta sus órdenes exclusivamente en simulación, lo que permite demostrar el ciclo completo de percepción, razonamiento y acción que caracteriza a un sistema agéntico manteniendo el capital real fuera del alcance del trabajo.

2.4 Sistemas web para gestión de inversiones

El desarrollo de plataformas tecnológicas para la gestión de inversiones requiere la integración de múltiples componentes de software que operen de manera coordinada y confiable. La arquitectura de estos sistemas debe contemplar la separación de responsabilidades entre el frontend, el backend, la capa de datos y los servicios externos, siguiendo principios de diseño que favorezcan la escalabilidad, el mantenimiento y la seguridad de la información financiera gestionada (Fowler, 2002).

2.4.1 Arquitectura cliente-servidor y APIs REST

La arquitectura cliente-servidor establece una separación entre la capa de presentación, ejecutada en el dispositivo del usuario, y la capa de lógica de negocio y datos, ejecutada en servidores remotos. Esta separación permite que múltiples clientes —interfaces web, aplicaciones móviles o sistemas de mensajería— interactúen con un mismo backend, favoreciendo la reutilización y la consistencia del sistema.

La comunicación entre cliente y servidor se realiza típicamente mediante APIs REST (*Representational State Transfer*), un estilo arquitectónico que define convenciones para el intercambio de datos a través del protocolo HTTP. Las APIs REST operan sobre recursos identificados por URLs, utilizando métodos estándar como GET, POST, PUT y DELETE para representar las operaciones de lectura, creación, actualización y eliminación de datos. Su naturaleza sin estado (*stateless*) facilita el escalamiento horizontal del backend, ya que cada solicitud contiene toda la información necesaria para ser procesada de forma independiente (Fielding, 2000).

En el presente proyecto, el backend está implementado en FastAPI, un framework de Python de alto rendimiento para la construcción de APIs REST. FastAPI incorpora validación automática de datos mediante tipado estático, generación automática de documentación interactiva y soporte nativo para operaciones asincrónicas, lo que resulta relevante en un sistema que gestiona simultáneamente solicitudes de usuarios, consultas a modelos de ML y comunicaciones con APIs externas.

2.4.2 Interfaces de usuario con React

El frontend del sistema está construido sobre React, una biblioteca de JavaScript desarrollada por Meta para la construcción de interfaces de usuario basadas en componentes reutilizables. React opera bajo un paradigma declarativo en el que el desarrollador describe el estado deseado de la interfaz y la biblioteca actualiza eficientemente el DOM del navegador cuando dicho estado cambia. Esta arquitectura, complementada con el framework de estilos utilitarios Tailwind CSS, permite construir interfaces responsivas y visualmente consistentes con un desarrollo ágil.

La aplicación frontend se despliega sobre Vercel, una plataforma de alojamiento optimizada para aplicaciones web modernas que ofrece integración directa con repositorios de GitHub, despliegue continuo automático ante cada actualización del código fuente y distribución global del contenido mediante una red de entrega de contenido (CDN). Esta integración garantiza que los usuarios accedan siempre a la versión más reciente del sistema con latencias mínimas.

2.4.3 Persistencia de datos con PostgreSQL y Supabase

La capa de persistencia está implementada sobre PostgreSQL, un sistema de gestión de bases de datos relacional de código abierto ampliamente utilizado en aplicaciones financieras por su robustez, soporte para transacciones ACID y capacidades avanzadas de consulta. PostgreSQL permite modelar con precisión las relaciones entre entidades como portafolios, movimientos, decisiones del agente y predicciones de los modelos ML, garantizando la integridad referencial de los datos.

El acceso a la base de datos se gestiona a través de Supabase, una plataforma de *backend as a service* construida sobre PostgreSQL que proporciona autenticación de usuarios, APIs autogeneradas, políticas de seguridad a nivel de fila (*Row Level Security*) y capacidades de tiempo real mediante *websockets*. Supabase simplifica la administración de la base de datos y la gestión de credenciales, permitiendo concentrar el esfuerzo de desarrollo en la lógica de negocio del sistema.

2.4.4 Despliegue e infraestructura en la nube

El backend del sistema se despliega sobre Railway, una plataforma de infraestructura en la nube que permite el despliegue de aplicaciones en contenedores aislados con configuración mínima. Railway gestiona automáticamente la provisión de recursos computacionales y el monitoreo del servicio, con integración directa a repositorios de GitHub para habilitar flujos de integración y despliegue continuo (CI/CD).

El agente de IA, en su modalidad de operación autónoma 24/7, se ejecuta sobre un servidor dedicado en Hostinger, manteniendo un proceso persistente con acceso continuo a los motores de lenguaje y a las APIs externas. Esta separación entre el servidor del agente y el backend de la aplicación responde a criterios de aislamiento de responsabilidades y gestión independiente de recursos. En conjunto, la arquitectura descrita —FastAPI en Railway, React en Vercel, PostgreSQL en Supabase y el agente en Hostinger— conforma un sistema distribuido en la nube donde cada componente puede escalarse, actualizarse y monitorearse de forma independiente, siguiendo los principios modernos de arquitectura de microservicios aplicados a la gestión financiera automatizada.

3. Arquitectura y metodología

3.1 Visión general del sistema

El sistema implementado se compone de cinco subsistemas coordinados: (i) una capa de **datos de mercado** que obtiene históricos diarios desde Yahoo Finance y cotizaciones en tiempo real; (ii) un **pipeline de machine learning** que entrena los cuatro modelos y genera predicciones diarias por activo; (iii) un **agente de IA** que consolida las predicciones en decisiones de rebalanceo explicadas en lenguaje natural; (iv) una capa de **ejecución simulada** que dimensiona y registra las órdenes resultantes vía Alpaca en modalidad *paper trading*; y (v) una **aplicación web** con un sitio público de documentación del proyecto y un portal operacional autenticado que expone portafolio, movimientos, decisiones del agente, rendimiento, predicciones y evaluación de modelos.

El flujo diario de operación está automatizado mediante tareas programadas de GitHub Actions: la generación de predicciones de los cuatro modelos, la decisión del agente sobre cada activo del universo y la revalorización del portafolio con precios de mercado (serie diaria de *performance* contra benchmark). Este diseño garantiza que la aplicación siempre exhibe datos frescos sin intervención manual, condición necesaria para un sistema que se declara autónomo.

El universo de inversión está compuesto por seis acciones listadas en mercados estadounidenses —AAPL, GOOGL, MSFT, NVDA, TSLA y SQM (ADR chileno, que aporta el nexo con el mercado local)— y tres referencias de mercado: el ETF SPDR S&P 500 (SPY), utilizado como *benchmark* del portafolio por corresponder al mismo mercado en que cotizan los activos; y, como

referencias comparativas del *backtesting*, el ETF iShares MSCI Chile (ECH) —proxy del IPSA dado que Yahoo Finance discontinuó la serie ^IPSA en 2019— y el índice Dow Jones (^DJI). El portafolio simulado se constituyó el primer día hábil de 2026 con compras valorizadas al precio de cierre real de esa fecha, y su evolución diaria se reconstruye y actualiza con cierres reales de mercado, de modo que toda cifra exhibida por el sistema es verificable contra fuentes públicas.

3.2 Datos y características

Los modelos se entrenan sobre cinco años de historia diaria por símbolo (precio de cierre ajustado por splits y dividendos, máximo, mínimo, apertura y volumen), descargados mediante la librería `yfinance`. Sobre la serie de precios se construye un conjunto de características técnicas comunes a los cuatro modelos: retornos simples y logarítmicos a distintos rezagos, medias móviles simples y sus razones, índice de fuerza relativa (RSI), volatilidad histórica por ventanas, momentum y *drawdown* acumulado. Las etiquetas dependen de la tarea: precio futuro a 5 días hábiles (regresión), dirección/señal a 5 días en tres clases —comprar, mantener, vender— definidas por umbrales sobre el retorno futuro, tendencia a 30 días y nivel de riesgo a 20 días en tres clases —bajo, medio, alto— definidas por la volatilidad y pérdida máxima futuras.

3.3 Configuración de los modelos

Cada modelo se especializa en la tarea para la cual su sesgo inductivo es más adecuado, y los cuatro comparten una interfaz común de entrenamiento y predicción que permite integrarlos de forma homogénea al pipeline:

Modelo	Tarea	Horizonte	Salida
LSTM (PyTorch)	Predicción de precio	5 días hábiles	Precio estimado + confianza (precisión direccional en validación)
XGBoost	Señal operativa	5 días hábiles	Clase {comprar, mantener, vender} + probabilidades
Prophet	Tendencia y estacionalidad	30 días	Dirección de tendencia + intervalo
Random Forest	Clasificación de riesgo	20 días	Clase {bajo, medio, alto} + probabilidades

Los clasificadores exponen además la distribución de probabilidad por clase, insumo necesario para el cálculo del AUC ROC multiclase descrito a continuación.

3.4 Metodología de evaluación: backtesting walk-forward sin look-ahead

El error metodológico más frecuente en la evaluación de modelos financieros es el sesgo de anticipación (*look-ahead bias*): entrenar o seleccionar el modelo usando información que no habría estado disponible en el momento de la predicción, lo que produce métricas ilusoriamente altas e irreplicables en operación real (López de Prado, 2018). Para eliminarlo, este proyecto adopta validación *walk-forward* con ventana expansiva: la historia de cada símbolo se divide en $K = 4$ pliegues temporales consecutivos; en el pliegue k , el modelo se entrena exclusivamente con datos anteriores al inicio del pliegue y predice, día a día, el período del pliegue; luego la ventana de entrenamiento se expande y el proceso se repite. Cada predicción evaluada se construyó, por diseño, solo con pasado — exactamente como operaría el sistema en producción. La partición aleatoria clásica queda descartada por mezclar pasado y futuro en series dependientes (Bergmeir y Benítez, 2012).

Para las tareas de regresión (LSTM, Prophet) se reportan RMSE, MAE, el error porcentual absoluto medio $MAPE = \frac{100}{n} \sum_t |(\hat{y}_t - y_t)/y_t|$ —que permite comparar activos de distinto nivel de precio— y la precisión direccional (proporción de días en que el signo del movimiento predicho coincide con el real), métrica económicamente más relevante que el error absoluto, pues una señal direccional correcta es accionable aunque el nivel predicho sea inexacto.

Para las tareas de clasificación (XGBoost, Random Forest) se reportan *accuracy*, F1 macro y, como métrica principal, el **área bajo la curva ROC (AUC ROC)**. La curva ROC grafica la tasa de verdaderos positivos contra la tasa de falsos positivos para todos los umbrales de decisión posibles; el área bajo ella equivale a la probabilidad de que el clasificador asigne mayor puntaje a una observación positiva elegida al azar que a una negativa elegida al azar (Fawcett, 2006). Un AUC de 50% corresponde a un clasificador aleatorio y 100% a discriminación perfecta. El AUC es preferible a la *accuracy* en este dominio por dos razones: es independiente del umbral de decisión —evalúa la calidad del ordenamiento probabilístico, no de un corte arbitrario— y es robusto al desbalance de clases, frecuente en etiquetas financieras donde la clase "mantener" o "riesgo medio" domina. Al tratarse de problemas de tres clases, se emplea la extensión One-vs-Rest con promedio macro: se calcula el AUC de cada clase contra el resto y se promedia sin ponderar, dando igual peso a las clases minoritarias (Fawcett, 2006; Hand y Till,

2001). El AUC se reporta en dos niveles deliberadamente: **global** (agregando las predicciones de todos los símbolos en una sola evaluación, $n = 600$ para Random Forest) y **por símbolo** ($n = 100$ cada uno), pues el primero mide la capacidad del modelo para ordenar el riesgo en todo el universo —incluida la discriminación entre regímenes de volatilidad de distintos activos— mientras que el segundo mide la discriminación dentro de cada serie individual, que es sistemáticamente más difícil.

Finalmente, la utilidad económica de las señales se evalúa con un *backtest* de estrategia: una regla *long-only* que toma posición cuando la señal de XGBoost indica compra y la liquida en señal de venta, comparada contra la referencia pasiva *buy & hold* del mismo universo sobre la misma ventana de evaluación. Se reportan retorno acumulado, razón de Sharpe anualizada $\text{Sharpe} = \sqrt{252} \bar{r} / \sigma_r$ (Sharpe, 1966) y pérdida máxima desde un peak (*maximum drawdown*), $\text{MDD} = \min_t (V_t / \max_{s \leq t} V_s - 1)$, que capturan las tres dimensiones relevantes: rentabilidad, retorno ajustado por riesgo y riesgo de cola.

3.5 El agente de IA: decisión determinista, explicación generativa

El diseño del agente responde a un principio rector: **en un dominio financiero, la decisión debe ser auditable; la creatividad del LLM se reserva para la comunicación**. El agente consolida las cuatro predicciones de cada activo en un puntaje unificado $s \in [-1, 1]$ mediante un voto direccional ponderado de los tres modelos predictivos (LSTM, XGBoost, Prophet), cuyo resultado se modula por el nivel de riesgo del Random Forest, que actúa como multiplicador de confianza: un activo clasificado como riesgo alto ve atenuada la magnitud de su puntaje y, con ello, el tamaño de cualquier orden resultante. Del puntaje se derivan de forma determinista la acción (comprar, vender, mantener o rebalancear), el nivel de confianza y una justificación estructurada. Este cálculo es una función pura de las predicciones: ante los mismos insumos produce siempre la misma decisión, lo que lo hace reproducible, testeable y defendible ante auditoría — propiedad que un LLM, por su naturaleza estocástica, no puede ofrecer.

Sobre esa decisión ya tomada, un modelo de lenguaje (Claude Haiku 4.5, de Anthropic, vía API) redacta la justificación en lenguaje natural claro y específico para el usuario final. El enrutador de LLM soporta múltiples motores (Anthropic, OpenAI, Ollama local) y degrada automáticamente a la justificación basada en reglas si el motor no está disponible, de modo que el pipeline de decisión nunca se interrumpe por una dependencia externa. Las preferencias del usuario se incorporan en la etapa de dimensionamiento: las órdenes de compra llevan la

posición hacia un peso objetivo del portafolio (15% por activo) escalado por la confianza de la decisión, y las de venta recortan la posición proporcionalmente, respetando los límites de exposición definidos.

La ejecución opera en modalidad *paper trading* mediante la API de Alpaca: las órdenes dimensionadas por el agente se envían al entorno de simulación del broker, que las valoriza con datos reales de mercado y las registra como movimientos del portafolio, cerrando el ciclo percepción–decisión–acción sin riesgo de capital. [PENDIENTE: la integración con Alpaca se encuentra en implementación; a la fecha las órdenes se registran como "intencionadas" con su dimensionamiento calculado, sin envío al broker.] La operación autónoma continua con notificaciones vía Telegram, orquestada por OpenClaw sobre un servidor dedicado, constituye la última fase de infraestructura del proyecto.

3.6 Plataforma web y operación continua

La aplicación web comprende dos áreas. El **sitio público** documenta el proyecto (problema, solución, arquitectura, modelos, resultados) con un diseño académico inspirado en publicaciones técnicas interactivas. El **portal operacional**, protegido por autenticación de Supabase con políticas RLS sobre todas las tablas operacionales, expone el estado vivo del sistema: composición y valorización del portafolio, historial de movimientos, decisiones del agente con su justificación y confianza, curva de rendimiento contra benchmark, predicciones diarias de los cuatro modelos por símbolo, un analizador en línea que permite consultar la inferencia de los modelos sobre cualquier símbolo, y una página de evaluación que publica las métricas del *backtesting* directamente desde la base de datos.

El backend expone la inferencia de los cuatro modelos como servicio (`GET /predict/{symbol}`) empaquetado en una imagen Docker con los artefactos de modelos versionados, desplegada en Railway. La calidad del código se resguarda con integración continua (lint y suite de 42 pruebas automatizadas en GitHub Actions), y tres tareas programadas diarias mantienen la frescura de los datos operacionales. Este grado de automatización sostiene la afirmación central del proyecto: el sistema funciona solo.

4. Resultados preliminares

Todos los resultados provienen del *backtesting walk-forward* descrito en 3.4 (4 pliegues con reentrenamiento, ventana expansiva, sin *look-ahead*), ejecutado sobre el universo de seis acciones más las referencias de contexto. Se presentan como resultados preliminares del anteproyecto; la versión final incorporará la operación acumulada del sistema en *paper trading*.

4.1 Predicción de precio (regresión)

Modelo	Horizonte	RMSE (USD)	MAE (USD)	MAPE	Precisión direccional
LSTM	5 días	10,35	7,61	4,2%	51,1%
Prophet	30 días	44,64	29,65	16,7%	53,5%

Valores promedio entre símbolos; RMSE y MAE dependen del nivel de precio de cada activo, por lo que el MAPE es la métrica comparable entre modelos y símbolos.

El LSTM logra un error porcentual bajo (4,2%) en el horizonte corto, coherente con su capacidad para explotar dependencia temporal reciente; su precisión direccional (51,1%), sin embargo, es apenas superior al azar, lo que confirma que predecir el *nivel* del precio a corto plazo es más tratable que predecir su *dirección*. Prophet, evaluado a 30 días, exhibe un error mayor —esperable al sextuplicar el horizonte— con una precisión direccional levemente mejor (53,5%), consistente con su diseño orientado a tendencia de mediano plazo.

4.2 Clasificación de señal y riesgo

Modelo	Tarea (3 clases)	Accuracy	F1 macro	AUC ROC global
XGBoost	Señal compra/venta (5 d)	41,1%	33,8%	53,6%
Random Forest	Nivel de riesgo (20 d)	59,7%	47,7%	78,2%

El contraste entre ambos clasificadores es el resultado más informativo del proyecto. La señal operativa de XGBoost alcanza un AUC global de 53,6%, estadísticamente cercano al clasificador aleatorio (50%): anticipar la dirección del retorno a 5 días a partir de indicadores técnicos es, en

la práctica, extremadamente difícil — un resultado alineado con la hipótesis de eficiencia débil del mercado, según la cual la información contenida en precios pasados ya está incorporada en el precio actual. Reportar este resultado con transparencia es parte del rigor metodológico del trabajo: un AUC alto en esta tarea habría sido, con alta probabilidad, síntoma de fuga de información.

El clasificador de riesgo de Random Forest, en cambio, alcanza un AUC ROC global de **78,2%** (One-vs-Rest macro, $n = 600$ predicciones, 4 pliegues): el modelo ordena correctamente el nivel de riesgo futuro en aproximadamente 78 de cada 100 comparaciones entre clases. La diferencia con la señal de XGBoost tiene fundamento financiero: la volatilidad es persistente y presenta *clustering* —períodos turbulentos tienden a seguir a períodos turbulentos—, de modo que el riesgo futuro es sustancialmente más predecible que la dirección del retorno. A nivel de símbolo individual el AUC del Random Forest fluctúa entre 44,4% (SQM) y 68,9% (AAPL), con promedio de 62,4%: la discriminación dentro de una misma serie es más difícil que en la evaluación agregada, donde el modelo también capitaliza las diferencias de régimen de volatilidad entre activos. Ambos niveles se reportan porque responden preguntas distintas — capacidad global de ordenamiento del riesgo en el universo versus capacidad de anticipar cambios de régimen en un activo específico.

4.3 Estrategia del sistema versus buy & hold

Estrategia	Retorno acumulado	Sharpe	Max drawdown
Sistema (long-only sobre señal XGBoost)	+139,0%	0,91	−40,0%
Buy & hold del universo	+218,7%	1,29	−31,0%

Misma ventana de evaluación (~200 días hábiles por símbolo, horizonte 5 días). Como contexto general del período histórico evaluado, los índices de referencia acumularon: ECH (proxy IPSA) +64,7%, Dow Jones +66,7% — calculados sobre la ventana completa de historia, no directamente comparables con la ventana de la estrategia.

La estrategia activa quedó por debajo de la referencia pasiva en retorno acumulado y en retorno ajustado por riesgo. El resultado es coherente con dos factores: primero, la ventana de evaluación coincidió con un período marcadamente alcista para el universo seleccionado (tecnológicas estadounidenses), régimen en el cual cualquier estrategia que permanezca

parcialmente fuera del mercado cede retorno frente a la exposición permanente; segundo, la señal que gatilla las posiciones (XGBoost, AUC 53,6%) carece de poder predictivo suficiente para compensar ese costo de oportunidad. Este resultado, lejos de debilitar el proyecto, valida su marco de evaluación: un *backtest* honesto y sin fuga de información produce conclusiones realistas — superar sistemáticamente al *buy & hold* en un mercado alcista es un estándar que la mayoría de la gestión activa profesional tampoco alcanza. Las conclusiones de diseño son directas: la señal direccional no debe usarse como gatillo único de ejecución, y el valor demostrable del sistema reside en la gestión de riesgo (donde el Random Forest sí discrimina, AUC 78,2%) y en la automatización integral del ciclo de gestión.

4.4 Sistema en producción

A la fecha de este anteproyecto, el sistema completo se encuentra desplegado y operativo: el backend sirve la inferencia de los cuatro modelos en producción, la aplicación web pública exhibe la documentación del proyecto y el portal operacional muestra datos vivos, el portafolio simulado se revaloriza diariamente con cierres reales de mercado (retorno acumulado desde inception el 2 de enero de 2026: +1,9% frente a +10,6% del benchmark S&P 500 —ETF SPY— al 6 de julio de 2026, un período en que el portafolio quedó por debajo del índice de su mercado), y las tres tareas programadas diarias (predicciones, decisiones del agente, valorización) llevan más de tres semanas ejecutándose de forma autónoma sin intervención manual. Las decisiones diarias del agente se registran con su justificación redactada por Claude Haiku 4.5 y su nivel de confianza, consultables en el portal.

5. Plan de trabajo restante

#	Actividad	Descripción	Estado
1	Ejecución paper trading	Integración del broker Alpaca (entorno <i>paper</i>): envío de las órdenes dimensionadas por el agente y registro de ejecuciones reales simuladas en el historial de movimientos	En curso
2	Operación autónoma 24/7	Despliegue del agente OpenClaw en servidor dedicado (Hostinger) con notificaciones proactivas vía Telegram	Pendiente
3	Acumulación de evidencia	Operación continua del sistema en <i>paper trading</i> para incorporar resultados de gestión reales al documento final	Continuo
4	Documento final y defensa	Redacción del informe final, láminas de presentación y ensayo de la demostración en vivo	Pendiente

[PENDIENTE: fechas comprometidas según calendario de defensa del programa.]

6. Conclusiones preliminares y trabajo futuro

Los resultados preliminares permiten extraer cuatro conclusiones. Primera, la **factibilidad técnica está demostrada**: un sistema de gestión de portafolio administrado por IA — modelos predictivos, agente de decisión, ejecución simulada y aplicación web— puede construirse e integrarse con herramientas de código abierto y servicios en la nube de costo acotado, y operar de forma autónoma con datos frescos diarios, como lo acredita el despliegue en producción.

Segunda, la evaluación comparativa de los modelos arroja una **asimetría con fundamento financiero**: el riesgo es sustancialmente más predecible que la dirección del retorno. El clasificador de riesgo (Random Forest, AUC ROC global 78,2%) constituye la señal más robusta del sistema, mientras la señal direccional de corto plazo (XGBoost, AUC 53,6%) se sitúa en el nivel del azar, en línea con la eficiencia débil del mercado. Esta asimetría orienta el diseño del agente: el riesgo modula el tamaño de las posiciones, y la señal direccional no debe operar como gatillo único.

Tercera, la comparación contra *buy & hold* en período alcista resultó desfavorable a la estrategia activa (+139,0% contra +218,7% acumulado), resultado que un marco de evaluación sin sesgo de anticipación hace explícito en lugar de ocultar. La contribución del trabajo no es una estrategia que "vence al mercado", afirmación que exigiría evidencia extraordinaria, sino un sistema íntegro, auditable y honesto de gestión automatizada.

Cuarta, la separación entre **decisión determinista** y **explicación generativa** —el agente calcula, el LLM redacta— demuestra ser un patrón de diseño apropiado para IA aplicada a finanzas: preserva la reproducibilidad y auditabilidad de las decisiones sin renunciar a la interacción en lenguaje natural que los LLM habilitan.

Como trabajo futuro se identifican: la incorporación de costos de transacción y deslizamiento al *backtest* de estrategia; el enriquecimiento del espacio de características con variables exógenas (sentimiento de noticias financieras procesado por LLM, indicadores macroeconómicos); la exploración de *ensembles* que combinen las señales de los cuatro modelos con ponderación aprendida; la evaluación del sistema en regímenes de mercado bajistas o laterales; y la extensión del universo de inversión a más activos y clases de activos.

Referencias bibliográficas

Bergmeir, C., y Benítez, J. M. (2012). On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191, 192–213.

Box, G. E. P., y Jenkins, G. M. (1976). *Time series analysis: Forecasting and control*. Holden-Day.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

Brown, T. B., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.

Chen, T., y Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.

- Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874.
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Tesis doctoral). University of California, Irvine.
- Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.
- Hand, D. J., y Till, R. J. (2001). A simple generalisation of the area under the ROC curve for multiple class classification problems. *Machine Learning*, 45(2), 171–186.
- Hochreiter, S., y Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- López de Prado, M. (2018). *Advances in financial machine learning*. Wiley.
- López-Lira, A., y Tang, Y. (2023). Can ChatGPT forecast stock price movements? Return predictability and large language models. *SSRN Working Paper*.
- Markowitz, H. (1952). Portfolio selection. *The Journal of Finance*, 7(1), 77–91.
- Narang, R. K. (2013). *Inside the black box: A simple guide to quantitative and high frequency trading*. Wiley.
- Russell, S., y Norvig, P. (2020). *Artificial intelligence: A modern approach* (4.^a ed.). Pearson.
- Salas, R. (2004). *Redes neuronales artificiales*. Universidad de Valparaíso, Departamento de Computación.
- Sharpe, W. F. (1966). Mutual fund performance. *The Journal of Business*, 39(1), 119–138.
- Taylor, S. J., y Letham, B. (2018). Forecasting at scale. *The American Statistician*, 72(1), 37–45.
- Ying, X. (2019). An overview of overfitting and its solutions. *Journal of Physics: Conference Series*, 1168, 022022.